

White-Box Testing

Project: Employee Management System in C++

1. Statement Coverage

Definition

Statement coverage checks whether every executable statement of the program runs at least once.

Applied on Code

In this project, statement coverage is applied on:

- isValidName()
- inputPositiveIdWithBack()
- inputSalaryWithBack()
- addemp()
- searchEmpById()
- removeempById()
- editiEmp()
- DisplayallEmp()

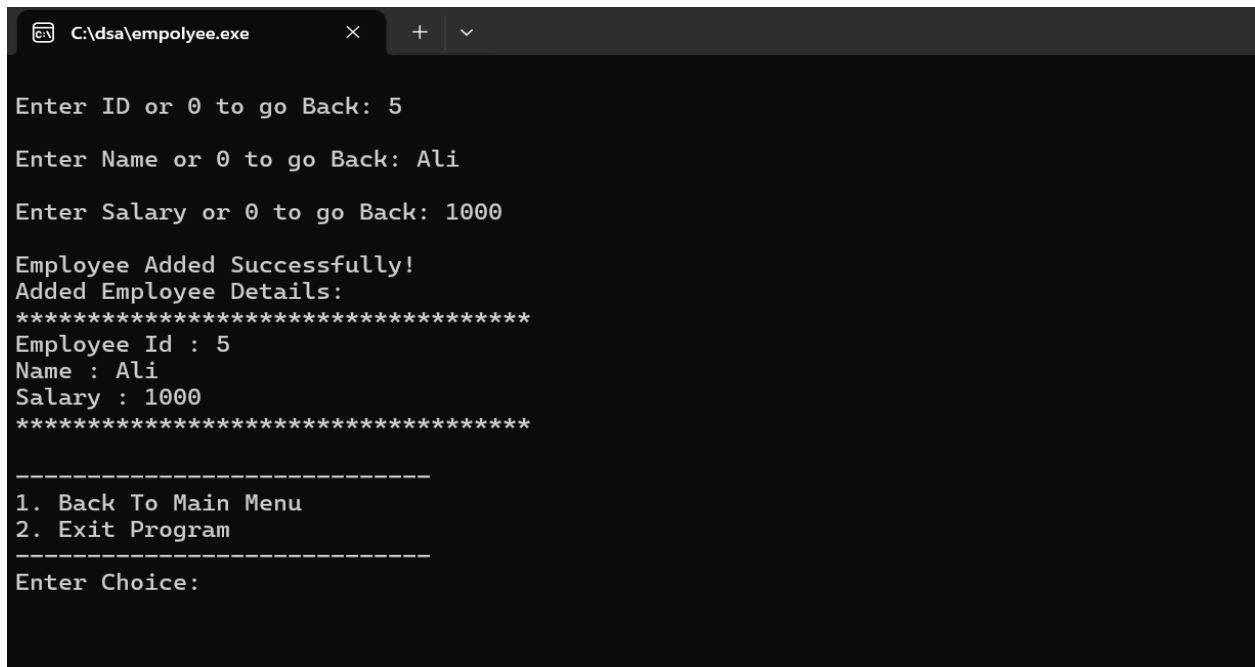
Test Cases

Test Case ID	Function	Input / Scenario	Expected Output
SC-01	isValidName()	Empty name	Invalid name
SC-02	isValidName()	Name greater than 15 letters	Invalid name
SC-03	isValidName()	Ali123	Invalid name
SC-04	isValidName()	Ali	Valid name
SC-05	inputPositiveIdWithBack()	abc	Characters not allowed
SC-06	inputPositiveIdWithBack()	-5	Negative ID not allowed
SC-07	inputPositiveIdWithBack()	5	ID accepted

SC-08	inputSalaryWithBack()	abc	Characters not allowed
SC-09	inputSalaryWithBack()	-2000	Negative salary not allowed
SC-10	inputSalaryWithBack()	30000	Salary accepted
SC-11	addemp()	ID 1, Name Ali, Salary 30000	Employee added successfully
SC-12	searchEmpById()	Existing ID	Employee found
SC-13	searchEmpById()	Non-existing ID	Employee not found
SC-14	removeempById()	Existing ID	Employee deleted
SC-15	editiEmp()	Update name and salary	Updated details shown
SC-16	DisplayallEmp()	Employees available	All employees displayed

Result

All important executable statements are executed at least once.



```

C:\dsa\empolyee.exe
Enter ID or 0 to go Back: 5
Enter Name or 0 to go Back: Ali
Enter Salary or 0 to go Back: 1000
Employee Added Successfully!
Added Employee Details:
*****
Employee Id : 5
Name : Ali
Salary : 1000
*****

-----
1. Back To Main Menu
2. Exit Program
-----
Enter Choice:

```

Statement Coverage Result: All selected major statements covered.

2. Branch Coverage

Definition

Branch coverage checks whether every true and false branch of decision statements is executed.

Applied on Code

Branch coverage is applied on:

- if
- else if
- else
- while
- switch

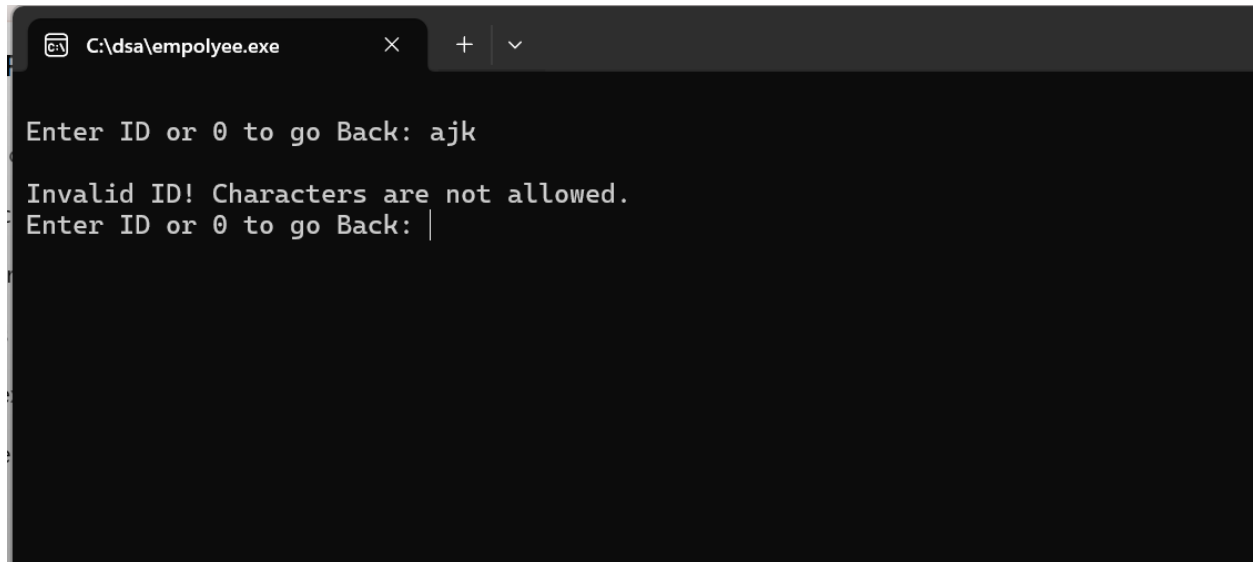
Test cases

Test Case ID	Condition / Branch	Input / Scenario	Expected Output
BC-01	name.length() == 0 true	Empty name	Invalid
BC-02	name.length() == 0 false	Ali	Continue checking
BC-03	name.length() > 15 true	Long name	Invalid
BC-04	name.length() > 15 false	Ali	Continue checking
BC-05	!isalpha(name[i]) true	Ali123	Invalid
BC-06	!isalpha(name[i]) false	Ali	Valid
BC-07	cin.fail() true	abc as ID	Invalid ID
BC-08	id < 0 true	-3	Negative ID rejected
BC-09	Valid ID branch	5	ID accepted
BC-10	start == NULL true	Empty list	No employee data
BC-11	start == NULL false	List has data	Continue operation
BC-12	ID match true	Search existing ID	Employee found
BC-13	ID match false	Search wrong ID	Employee not found
BC-14	Switch case 1	Choice 1	Add employee
BC-15	Switch case 2	Choice 2	Display employee

BC-16	Switch case 3	Choice 3	Search employee
BC-17	Switch case 4	Choice 4	Delete employee
BC-18	Switch case 5	Choice 5	Edit employee
BC-19	Switch case 6	Choice 6	Exit program

Result

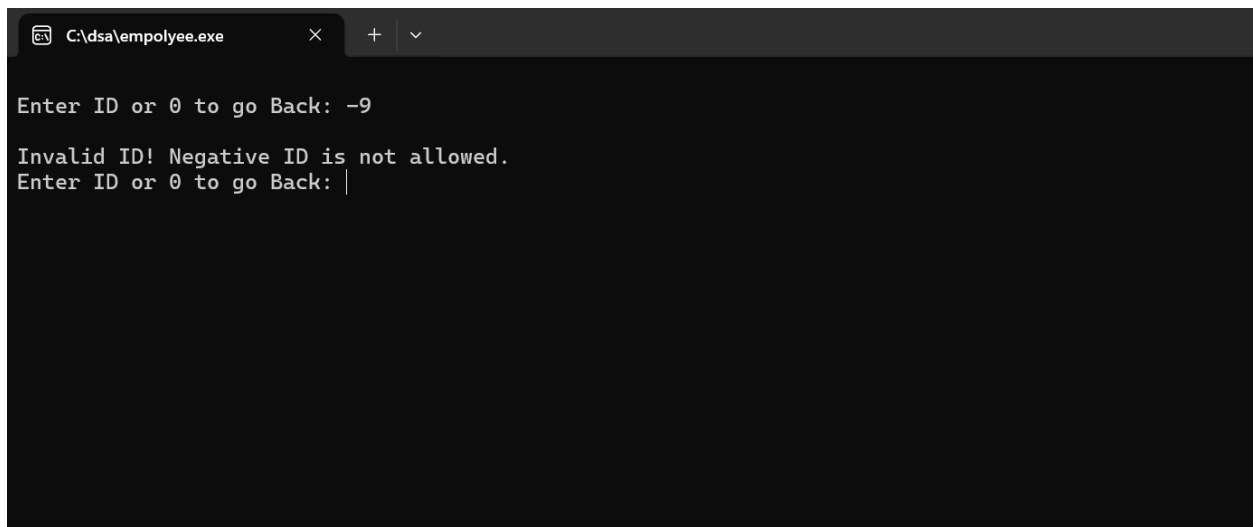
All true and false branches are covered.



```

C:\dsa\empolyee.exe
Enter ID or 0 to go Back: ajk
Invalid ID! Characters are not allowed.
Enter ID or 0 to go Back: |

```



```

C:\dsa\empolyee.exe
Enter ID or 0 to go Back: -9
Invalid ID! Negative ID is not allowed.
Enter ID or 0 to go Back: |

```

Branch Coverage Result: 100%

3. Condition Coverage

Definition

Condition coverage checks each individual Boolean condition with both true and false values.

Important Conditions

Condition ID	Condition	True Input	False Input
C1	<code>name.length() == 0</code>	Empty name	Ali
C2	<code>name.length() > 15</code>	Long name	Ali
C3	<code>!isalpha(name[i])</code>	Ali123	Ali
C4	<code>cin.fail()</code>	abc	5
C5	<code>id < 0</code>	-3	5
C6	<code>sal < 0</code>	-2000	30000
C7	<code>ans == 'Y' ans == 'y'</code>	Y	N
C8	<code>start == NULL</code>	Empty list	List has data
C9	<code>p != NULL</code>	Node exists	End of list
C10	<code>ch < 1 ch > 6</code>	0 or 7	1 to 6

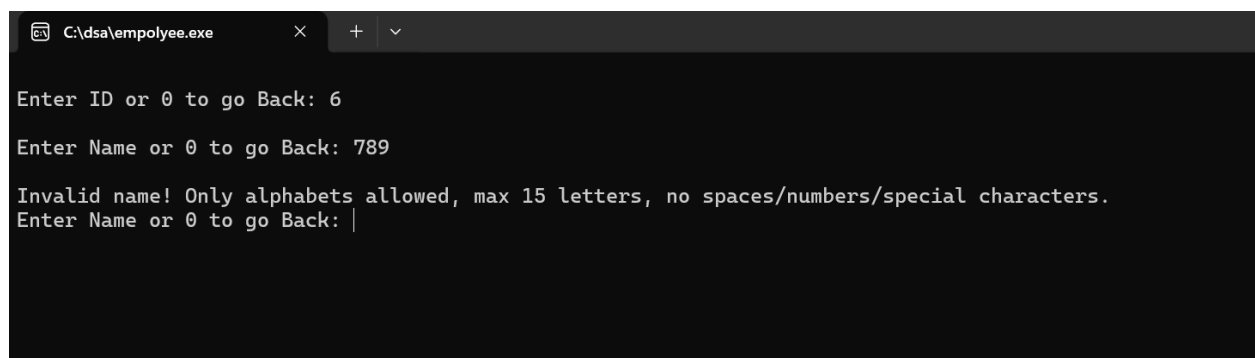
Test Cases

Test Case ID	Input / Scenario	Condition Covered	Expected Output
CC-01	Empty name	C1 true	Invalid
CC-02	Ali	C1 false, C2 false, C3 false	Valid
CC-03	Long name	C2 true	Invalid

CC-04	Ali123	C3 true	Invalid
CC-05	ID abc	C4 true	Invalid
CC-06	ID -3	C5 true	Invalid
CC-07	ID 5	C4 false, C5 false	Valid
CC-08	Salary -2000	C6 true	Invalid
CC-09	Salary 30000	C6 false	Valid
CC-10	Answer Y	C7 true	Update allowed
CC-11	Answer N	C7 false	Update skipped
CC-12	Empty list	C8 true	No data
CC-13	Existing employee	C8 false, C9 true	Operation continues
CC-14	Menu choice 7	C10 true	Invalid choice
CC-15	Menu choice 3	C10 false	Search selected

Result

Each important condition is tested with true and false values.



```

C:\dsa\empolyee.exe
Enter ID or 0 to go Back: 6
Enter Name or 0 to go Back: 789
Invalid name! Only alphabets allowed, max 15 letters, no spaces/numbers/special characters.
Enter Name or 0 to go Back: |

```

Condition Coverage Result: 100%

4. Path Coverage

Definition

Path coverage checks different possible execution paths of a function from start to end.

Path Coverage for isValidName()

Path ID	Input	Path Covered	Expected Output
PC-01	Empty name	Start → empty check true → return false	Invalid
PC-02	Long name	Start → empty false → length > 15 true → return false	Invalid
PC-03	Ali123	Start → valid length → loop → non-alpha true → return false	Invalid
PC-04	Ali	Start → valid length → loop all alphabet → return true	Valid

```
C:\dsa\employee.exe

Enter ID or 0 to go Back: 5
Enter Name or 0 to go Back: Ali
Enter Salary or 0 to go Back: 1000

Employee Added Successfully!
Added Employee Details:
*****
Employee Id : 5
Name : Ali
Salary : 1000
*****

-----
1. Back To Main Menu
2. Exit Program
-----
Enter Choice:
```

Path Coverage for searchEmpById()

Path ID	Scenario	Path Covered	Expected Output
PC-05	Empty list	Start → start == NULL → return	No employee data

PC-06	ID found at first node	Start → list not empty → first ID matched → display → return	Employee found
PC-07	ID found after first node	Start → first ID not matched → next ID matched → display → return	Employee found
PC-08	ID not found	Start → traverse complete list → not found	Employee not found

```

C:\dsa\empolyee.exe  X  +  v

Enter ID or 0 to go Back: 1

Employee Found.
*****
Employee Id : 1
Name : Ali
Salary : 90
*****

-----
1. Back To Main Menu
2. Exit Program
-----
Enter Choice: |

```

```

C:\dsa\empolyee.exe  X  +  v

Enter ID or 0 to go Back: 33

Employee is not found.

-----
1. Back To Main Menu
2. Exit Program
-----
Enter Choice: |

```

Path Coverage for removeempById()

Path ID	Scenario	Path Covered	Expected Output
PC-09	Empty list	Start → start == NULL → return	No employee data
PC-10	Delete first employee	Start → first ID matched → delete first node → return	Deleted
PC-11	Delete second/last employee	Start → first not matched → loop → next matched → delete → return	Deleted
PC-12	ID not found	Start → traverse list → no match → not found	Employee not found

```
C:\dsa\empolyee.exe  ×  +  ∨

Enter ID or 0 to go Back: 1

*****
Employee Id : 1
Name : Ali
Salary : 90
*****
Employee Deleted Successfully.
```

```
C:\dsa\empolyee.exe  ×  +  ∨

Enter ID or 0 to go Back: 44

No employee data is here.

-----
1. Back To Main Menu
2. Exit Program
-----
Enter Choice: |
```

Result

Important independent paths of validation, search, and delete functions are covered.

Path Coverage Result: High / Practical 100%

5. Loop Testing

Definition

Loop testing checks loops with zero, one, two, and multiple iterations.

Loops in Project

Loop ID	Function	Loop Purpose
L1	isValidName()	Check each character
L2	inputPositiveIdWithBack()	Repeat until valid ID
L3	inputSalaryWithBack()	Repeat until valid salary
L4	inputYesNo()	Repeat until Y/N
L5	isIdExist()	Traverse linked list
L6	searchEmpById()	Traverse linked list
L7	removeempById()	Traverse for deletion
L8	DisplayallEmp()	Display all employees
L9	main()	Menu repetition

Test Cases

Test Case ID	Loop	Scenario	Iteration Type	Expected Output
LT-01	isValidName()	Empty name	0 iteration	Invalid
LT-02	isValidName()	A	1 iteration	Valid

LT-03	isValidName()	Ali	Multiple iterations	Valid
LT-04	ID input loop	5	1 iteration	Accepted
LT-05	ID input loop	abc, then 5	2 iterations	Invalid then accepted
LT-06	ID input loop	abc, -3, 5	Multiple iterations	Invalid until valid
LT-07	Salary input loop	30000	1 iteration	Accepted
LT-08	Salary input loop	abc, -100, 30000	Multiple iterations	Invalid until valid
LT-09	inputYesNo()	Y	1 iteration	Accepted
LT-10	inputYesNo()	x, then N	2 iterations	Invalid then accepted
LT-11	Linked list traversal	Empty list	0 iteration	No data / false
LT-12	Linked list traversal	One employee	1 iteration	Found/displayed
LT-13	Linked list traversal	Three employees	Multiple iterations	Traversal completed
LT-14	Main menu loop	Choice 6 directly	1 iteration	Exit
LT-15	Main menu loop	Choices 1, 2, 6	Multiple iterations	Add, display, exit

Result

Loops are tested for zero, one, two, and multiple iterations.

```
C:\dsa\empolyee.exe  X  +  v

Enter ID or 0 to go Back: abc

Invalid ID! Characters are not allowed.
Enter ID or 0 to go Back: -6

Invalid ID! Negative ID is not allowed.
Enter ID or 0 to go Back: 1

Enter Name or 0 to go Back: 90

Invalid name! Only alphabets allowed, max 15 letters, no spaces/numbers/special characters.
Enter Name or 0 to go Back: Ahmad

Enter Salary or 0 to go Back: |
```

Loop Testing Result: 100%

6. Basis Path Testing

Definition

Basis path testing identifies independent paths using cyclomatic complexity.

Formula:

Cyclomatic Complexity = Predicate Nodes + 1

Basis Path Testing for removeempById()

Predicate Nodes

Predicate No.	Decision
P1	if (start == NULL)
P2	if (id == p->getdata().getid())
P3	while (p->getnext() != NULL)
P4	if (id == p->getnext()->getdata().getid())

Cyclomatic Complexity:

$$V(G) = P + 1$$

$$V(G) = 4 + 1$$

$$V(G) = 5$$

```

C:\dsa\empolyee.exe
Enter ID or 0 to go Back: 1

*****
Employee Id : 1
Name : Ali
Salary : 90
*****
Employee Deleted Successfully.

```

```

C:\dsa\empolyee.exe
Enter ID or 0 to go Back: 44

No employee data is here.

-----
1. Back To Main Menu
2. Exit Program
-----
Enter Choice: |

```

Independent Paths

Path ID	Scenario	Test Data	Expected Output
BP-01	Empty list	Delete ID 1 from empty list	No employee data
BP-02	Delete first node	List: ID 1, delete 1	Employee deleted
BP-03	Delete second node	List: ID 1, 2; delete 2	Employee deleted
BP-04	Delete last node after multiple traversal	List: ID 1, 2, 3; delete 3	Employee deleted
BP-05	ID not found	List: ID 1, 2; delete 5	Employee not found

Basis Path Testing for editiEmp()

Predicate Nodes

Predicate No.	Decision
P1	if (start == NULL)
P2	while (p != NULL)
P3	if (p->getdata().getid() == id)

P4	if (ans == 'Y' ans == 'y') for name
P5	if (name == "0")
P6	else if (!isValidName(name))
P7	else if (isNameExistExceptCurrentId(name, id))
P8	if (ans == 'Y' ans == 'y') for salary
P9	if (sal == 0)

Cyclomatic Complexity:

$$V(G) = P + 1$$

$$V(G) = 9 + 1$$

$$V(G) = 10$$

```

C:\dsa\empolyee.exe

Enter ID or 0 to go Back: 1

Do you want to change name? (Y/N): n

Do you want to change Salary? (Y/N): y

Enter Salary or 0 to go Back: 87876

Employee Data Updated Successfully!
Updated Employee Details:
*****
Employee Id : 1
Name : Ahmad
Salary : 87876
*****

-----
1. Back To Main Menu
2. Exit Program
-----
Enter Choice: |

```

```
C:\dsa\empolyee.exe

Enter ID or 0 to go Back: 55

Record not found.

-----
1. Back To Main Menu
2. Exit Program
-----
Enter Choice: |
```

Independent Paths

Path ID	Scenario	Test Data	Expected Output
BP-06	Empty list	Edit ID 1 from empty list	No Employee is here
BP-07	ID not found	List has ID 1, edit ID 5	Record not found
BP-08	No update selected	Existing ID, Name: N, Salary: N	Same data saved again
BP-09	Valid name update only	Y, new name Sara, Salary: N	Name updated
BP-10	Name update cancelled	Y, name 0, Salary: N	Name update cancelled
BP-11	Invalid name then valid	Y, Ali123, then Sara	Invalid then accepted
BP-12	Duplicate name then valid	Y, existing name, then unique name	Duplicate rejected

BP-13	Salary update only	Name: N, Salary: Y, 40000	Salary updated
BP-14	Salary update cancelled	Name: N, Salary: Y, 0	Salary cancelled
BP-15	Both name and salary updated	Y, Sara, Y, 50000	Both updated

Result

Independent paths are tested according to cyclomatic complexity.

Basis Path Testing Result: Covered